COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/06/2026	**DATE SUBMITTED:**	
NAME:	Reyes, Eli Justin L. Santos, Adrian Walter D.	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A INPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

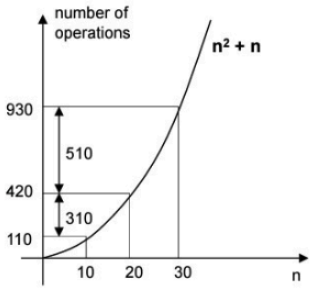
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis	The algorithm doesn't handle bigger tasks well, because based on the illustration shown on the graph, the line curves dramatically exponentially upward. Every time you add 1 to n, the number of steps it has to take jumps way up based on $n^2 + n$. This shows just how inefficient the algorithm is, since even small increases in input make it run super slow.

ILO B INPUT

main.cpp > ...

```cpp
#include <iostream>
#include "header.h"

int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}

```

header.h > generate_random_Student(int)

```cpp
#ifndef HEADER_H
#define HEADER_H

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student {
    private:
    std::pair<int, int> name;
    bool vaccinated;

    public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {
    }

    // Getters
    auto get_name() const { return name; }
    auto is_vaccinated() const { return vaccinated; }

    // Two people are same if they have the same name
    bool operator ==(const Student& p) const {
        return this->name == p.name;
    }
    // The ordering of a set of people is defined by their name
    bool operator< (const Student& p) const {
        return this->name < p.name;
    }
    bool operator> (const Student& p) const {
        return this->name > p.name;
    }
};

// Forward declaration required so search_test can call needs_vaccination
bool needs_vaccination(Student P, const std::vector<Student>& people);

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
```

```
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));    class "std::vector<Student, std::allocator<Student>>" has no member "push_back"
    }

    std::sort(people.begin(), people.end());    class "std::vector<Student, std::allocator<Student>>" has no member "begin"

    // To measure the time taken, start the clock    'needs_vaccination' was not declared in this scope
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") " << "needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") " << "does not need vaccination." << std::endl;
    }
}

bool needs_vaccination(Student P, const std::vector<Student>& people) {
    auto first = people.begin();    class "std::vector<Student, std::allocator<Student>>" has no member "begin"
    auto last = people.end();    class "std::vector<Student, std::allocator<Student>>" has no member "end"

    while (first < last) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);

```

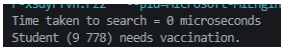
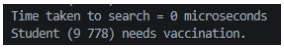
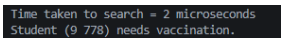
```
94          if (mid_element == P) {
95              // Found student: returns true if NOT vaccinated
96              return !mid_element.is_vaccinated();
97          }
98          else if (mid_element > P) {
99              // Search lower half: exclude mid_element
100             last = first + mid_element_index;
101         }
102         else {
103             // Search upper half: exclude mid_element
104             first = first + mid_element_index + 1;
105         }
106     }
107
108     // Student not found in the sequence -> needs vaccination
109     return true;
110 }
111
112 #endif
```

Code Analysis	So the code begins with overloading the operators for the student class so it can be sorted, then functions to generate students from random data, followed by code to run and test the outputs, and finally a function that implements our logic for whether a vaccination is needed (bool needs_vaccination). The main.cpp is to hold our driver code for everything in the header file.

ILO B OUTPUT

```
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1 - REYES\HOA11.1>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\TIPQC\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-qylhgkok.qau' '--stdout=Microsoft-MIEngine-Out-uezseg0o.jzy' '--stderr=Microsoft-MIEngine-Error-cnzdky1m.5w0' '--pid=Microsoft-MIEngine-Pid-jpkghikr.mnj' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Time taken to search = 0 microseconds
Student (543 434) needs vaccination.
Time taken to search = 1 microseconds
Student (543 434) needs vaccination.
Time taken to search = 1 microseconds
Student (543 434) needs vaccination.
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1 - REYES\HOA11.1>
```

Conclusion	

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 microsecond		Out of 9,778, this process was almost instant.
10000	0 microsecond		Logically, this process would be a decimal slower than the one with 1000 input size.
100000	2 microsecond		This took 2 microseconds, therefore it is the longest out of the three.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.